

Self-attention: match, mix, project, add

A token keeps its current representation and adds information read from allowed source tokens.

Q and K choose the weights. Those weights mix V. The output projection makes an update that can be added to E.

1 • Three roles, not three copies

Query q_i : what the receiving token is looking for.

Key k_j : features used to decide whether source j is a useful match.

Value v_j : information source j contributes if read.

Directory analogy: query = “contact the library”; key = “Library”; value = {extension 204, room 2B, hours 09:00-18:00}. The matching name does not supply the contact details.

Real attention uses numeric vectors, not literal names or records. The analogy explains the roles; a neural value is a learned representation of useful information.

2 • First choose where to read

Receiver i compares with source j :

$$s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}.$$

Normalize over the **allowed sources**:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{r \leq i} \exp(s_{ir})}, \quad j \leq i.$$

Set future weights to zero. Each allowed weight is nonnegative; each row sums to 1. These weights depend on Q and K, not V.

3 • Then mix what sources send

$$m_i = \sum_{j \leq i} \alpha_{ij} v_j.$$

One scalar weight multiplies **every coordinate** of its source’s value. All values share width d_v , so they can be added.

Hard retrieval returns one winner’s value. Soft attention combines values using a distribution of weights. Changing only V leaves the weights fixed but can change the message.

4 • Follow the lecture’s bank example

Prefix: **The fisherman sat beside the river bank.**

Receiver is **bank** at $i = 7$. In the hand-chosen teaching model, $d_{\text{model}} = 4$, $d_k = 3$, $d_v = 2$.

$$e_7 = [0.70, 0.70, 0.10, 0.80]$$

$$q_7 = [1.26, 1.26, 0.04].$$

e axes: water, finance, person, glue. Q/K axes: water?, finance?, who? Value axes: water scene, finance scene.

Source	Weight	Value row
The	0.050	(0.1, 0.0)
fisherman	0.226	(2.0, 0.1)
sat	0.059	(0.3, 0.0)
beside	0.067	(0.6, -0.1)
the	0.054	(0.1, 0.1)
river	0.414	(3.1, -0.1)
bank	0.130	(0.7, 0.7)

Multiply each row by its weight, then add:

$$m_7 \approx [1.89, 0.07].$$

For example, river contributes about $0.414 \times [3.1, -0.1] = [1.283, -0.041]$.

5 • Project before the residual addition

A two-number message cannot be added to a four-number representation. A learned map changes the coordinates and width:

$$W_O = \begin{pmatrix} 0.8 & -0.2 & 0.3 & 0.1 \\ -0.1 & 1.1 & 0.2 & -0.2 \end{pmatrix}.$$

$$\Delta e_7 = m_7 W_O \approx [1.51, -0.30, 0.58, 0.18].$$

$$e_7^{\text{new}} = e_7 + \Delta e_7$$

$$\approx [2.21, 0.40, 0.68, 0.98].$$

This is a **linear mixing operation**, not zero-padding or copying coordinates. “Projection” here does not mean an orthogonal projection. A bias can be included; this toy omits it.

Values and weights are rounded for display. The toy’s water/finance/person/glue labels are teaching aids, not a claim that trained models have named axes.

Where Q, K and V come from

Row-vector convention • T token positions • one attention head • biases omitted in the attention equations.

6 • Same matching width, different jobs

Each current row e_i supplies all three vectors:

$$q_i = e_i W_Q, \quad k_i = e_i W_K, \quad v_i = e_i W_V.$$

Q and K must share width d_k for the dot product. **Equal shape does not require equal entries or equal mappings.**

For “Maya grabbed a coat because she was cold,” a query from **she** may ask for a person. Maya’s key may advertise a person-like source; its value may contribute identity and number information.

W_Q extracts what a token seeks; W_K extracts what it can match; W_V extracts what it sends. Tying the matrices is possible but restricts these roles. Each matrix is shared across positions in the head.

7 • The complete matrix path

Object

current rows E

W_Q, W_K

Q, K

W_V

V , message AV

scores S , mask M ,

weights A

W_O

Shape

$T \times d_{\text{model}}$

$d_{\text{model}} \times d_k$

$T \times d_k$

$d_{\text{model}} \times d_v$

$T \times d_v$

$T \times T$

$d_v \times d_{\text{model}}$

$$S = Q \frac{K^\top}{\sqrt{d_k}}$$

$$A = \text{softmax}_{\text{row}}(S + M)$$

$$\Delta E = (AV)W_O, \quad E^{\text{new}} = E + \Delta E.$$

Residual route: keep E unchanged until the addition.

Attention route: compute its update. Both meet in the same representation space.

8 • The next-token predictor still has an MLP

In this lecture’s toy model, the final position uses

$$h = \text{ReLU}(e_T^{\text{new}} W_1 + b_1)$$

$$z = h W_2 + b_2, \quad p = \text{softmax}(z).$$

Widths: **4 → 8 hidden units → 20 logits**. Even **the** can ask a broad query; the preceding keys and values supply context-specific information. The toy prediction head is not the full Transformer architecture.

9 • Scaling controls score spread

Assume query and key coordinates are mutually independent, mean 0, variance 1. Then

$$\text{Var}(q_i k_i) = 1$$

$$\text{Var}(q \cdot k) = d_k, \quad \text{SD}(q \cdot k) = \sqrt{d_k}.$$

Variances of independent products add. Dividing the score by $\sqrt{d_k}$ gives unit SD under these assumptions.

Entries **need not be ± 1** : those make an easy simulation; independent standard-normal coordinates also satisfy the assumptions. Learned vectors need not satisfy them exactly.

For $d_k = 64$, suppose scores are $[-8, 8]$:

$$p_A = \frac{\exp(-8)}{\exp(-8) + \exp(8)} \approx 0.000000113.$$

After division by 8, the scores are $[-1, 1]$:

$$p_A = \frac{\exp(-1)}{\exp(-1) + \exp(1)} \approx 0.1192.$$

$p_B = 1 - p_A$. Scaling avoids excessively sharp weights and tiny softmax derivatives. **Overflow is separate:** subtracting the maximum is numerically stable but does not reduce score gaps.

10 • A causal mask blocks the answer

At position i , predict token $i + 1$. Read positions $j \leq i$, including the current token, but never future positions.

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

S holds scaled match scores. **M** adds zero to allowed scores and $-\infty$ to forbidden scores. **A** is the resulting row-wise probability distribution.

Example at position 2, with four sources:

$$[1, 2, 3, 4] + [0, 0, -\infty, -\infty]$$

$$\rightarrow \text{softmax} \rightarrow [0.269, 0.731, 0, 0].$$

Since $\exp(-\infty) = 0$, future values have zero weight. Apply the mask **before** softmax, not after it without renormalizing.

Training: the full sequence is available, but each position must not see its answer. **Generation:** future tokens have not been generated yet. The same causal rule keeps the two settings consistent.